Calcular π sin calcular los dígitos anteriores

La fórmula BBP en MapReduce, con los ejemplos bbp y distbbp

Equipo 5, Big Data Analytics. Hadoop 3.3.6 en modo standalone, dentro de un contenedor Debian.

Alejandro Jiménez Reyes, Elías Francisco Maltrana Luna y Daniel Ruvalcaba Juárez

github.com/Alejandro-Jimenez-Reyes/bda-equipo-5-pi-bbp

¿Qué hacen estos dos ejemplos?

bbp

Saca dígitos hexadecimales de π desde la posición que le pidas, hasta unos 10^8. Corre un solo job de MapReduce. Si empiezas en el dígito 1, también escribe un pi.txt con el resultado en hex y en decimal.

distbbp

Saca bits de π a partir del bit b+1, pensado para posiciones enormes (de 10^8 en adelante). Lanza muchos jobs y guarda resultados parciales, así que se puede retomar si se corta.

Lo que nos interesa

- Con BBP se llega al dígito n de π en base 16 sin haber calculado los n−1 anteriores.
- Por eso el trabajo se puede repartir entre nodos: cada uno hace su tramo y no espera a los demás.
- Un caso real: comprobar un récord de π calculando un dígito muy lejano con otro método, en un clúster, porque la carga es pesada y desigual.

La matemática, en una línea

$$\pi = \Sigma_{k=0..\infty}\ 16^{-k} \cdot \left(4/(8k+1) - 2/(8k+4) - 1/(8k+5) - 1/(8k+6) \right)$$

- Como $16^{-k} = 2^{-4k}$, al multiplicar por 16^{d} los dígitos anteriores a la posición d quedan en la parte entera, y nos quedamos solo con la fraccionaria.
- Para el dígito d+1 sumamos los términos con $k \le d$ usando aritmética modular ($16^{d-k} \bmod (8k+j)$) y unos cuantos términos más.
- Cada llamada nos da 4 dígitos hex. bbp calcula 4 sumas: S1, S4, S5 y S6.
- distbbp usa otra fórmula parecida, la de Bellard (1997), que es más eficiente. Sus sumas se parten en trabajos.

bbp: cómo se reparten Map y Reduce

Parámetros

startDigit, nDigits y nMaps. No hay archivo de entrada.

↓

BbpInputFormat

Crea nMaps splits (offset, longitud) repartidos según la carga.

MAP 1 a MAP 4

Cada map recibe su (offset, len) y calcula sus dígitos hex con las 4 sumas.

↓ clave = offset, valor = bytes con los dígitos

REDUCE (uno solo)

Junta los trozos en orden de offset y escribe en workingDir pi_1_100.hex y, solo si startDigit = 1, pi.txt.

Los dígitos lejanos cuestan más, así que los maps no reciben el mismo número de dígitos.

Restricciones: nDigits múltiplo de 4, startDigit ≥ 1 y el directorio de trabajo no puede existir ya.

Ejemplo: bbp 1 100 4 ...

distbbp: muchos jobs de MapReduce

1. Cliente DistBbp

Lee b y arma las sumas de Bellard (hasta 14).

↓

2. Sumas

Cada suma se parte en nJobs trabajos.

↓

3. Jobs MapReduce

Hasta nThreads a la vez, cada uno dividido en nPart partes.

type: m, r o x

m = la suma va en el map. r = en el reduce. x = mezcla de ambos.

↓

Resultados parciales

Cada TaskResult se guarda en localDir.

↓

Combinar

Suma todo módulo 1 y queda la fracción de $\pi \cdot 2^b$.

Reanudar

Si se interrumpe, repetimos el comando con otro remoteDir. Lo que ya está en localDir no se recalcula.

```
b = 1,000 (bits skipped)
574E69A458FF 6 (12 hex digits)    # salida real, el último dígito puede variar ±1
```

Parámetros: <b> <nThreads> <nJobs> <type> <nPart> <remoteDir> <localDir>. En standalone todo es disco local.

Ejemplo: distbbp 1000 2 2 m 2 ...

bbp frente a distbbp

	bbp	distbbp
Posiciones	hasta ≈ 10^8 (dígitos hex)	desde 10^8 (bits)
Jobs MapReduce	1	decenas o cientos
Reduce	1 reducer que concatena	opcional (tipo r o x)
Fórmula	BBP (4 sumas)	Bellard (hasta 14 sumas)
Salida	pi.txt y archivo .hex	consola y parciales en localDir
Reanudable	no	sí, cambiando remoteDir
Parámetros	4	7

Frente al Equipo 4 (pi con Monte Carlo): ellos obtienen una aproximación aleatoria que mejora un poco con más muestras. BBP es determinista y exacto, cada dígito sale correcto y no interviene el azar.

Demo en vivo

```
# en nuestra computadora
docker start hadoop
bash scripts/00_copiar_al_contenedor.sh
docker exec -it hadoop bash

# dentro del contenedor
bash /tmp/bbp/scripts/ejecutar.sh
```

```
# bbp: dígitos hex 1 a 100, 4 maps
hadoop jar $JAR bbp 1 100 4 $WORK/bbp_1_100
cat $WORK/bbp_1_100/pi.txt

# bbp: dígitos 1001 a 1020 (saltando 1000)
hadoop jar $JAR bbp 1001 20 2 $WORK/b2
# distbbp con b = 100
hadoop jar $JAR distbbp 100 2 2 m 2 R L
```

- Primero listamos los ejemplos con hadoop jar $JAR y aparecen bbp y distbbp.
- bbp exige un directorio de trabajo nuevo: si repetimos el comando falla, y es a propósito.
- Abrimos el .hex con od: cada byte son 2 dígitos hex de π.
- Comparamos contra datos/referencia_pi_hex.txt.

Resultados que obtuvimos

```
Pi = 0x3.243F6A8885 A308D31319 8A2E037073 44A4093822 299F31D008
        2EFA98EC4E 6C89452821 E638D01377 BE5466CF34 E90C6CC0AC
Total number of hexadecimal digits is 100.

Pi = 3.1415926535 8979323846 2643383279 5028841971 6939937510
      5820974944 5923078164 0628620899 8628034825 3421170679
Total number of decimal digits is 100.
```

bbp 1001 20 2

Dígitos hex 1001 a 1020:

`49F1C09B075372C98099`

distbbp

`b=100 → 3707344A4093 E`

`b=1000 → 574E69A458FF 6`

Es la salida real de Hadoop. El valor exacto para b=1000 es 574E69A458FE A: el 12.º dígito varía ±1 por error de coma flotante.

Ejercicio para ustedes

- **A)** Con bbp, calculen los dígitos hexadecimales 101 a 120 de π con 2 maps, en el directorio /tmp/ej_bbp.
- **B)** Con distbbp, b = 100, 2 hilos, 2 jobs, tipo m y 2 partes: ¿en qué dígito hex empieza el resultado?

Para pensar

- ¿Por qué en A no se genera pi.txt?
- ¿Qué error da bbp si nDigits = 10, y por qué?
- ¿Por qué los maps de bbp reciben tramos de distinto tamaño?
- ¿Qué pasa si repiten el comando de A sin cambiar el directorio?

El enunciado completo está en ejercicio/ejercicio.md y la solución en ejercicio/solucion.md. Con el README del repositorio se puede reproducir todo.

Conclusiones

- BBP da los dígitos de π en base 16 sin los anteriores, y eso es lo que permite repartir el trabajo.
- bbp usa 1 job, 4 sumas y 1 reducer que concatena. distbbp usa muchos jobs con las sumas de Bellard y se puede reanudar.
- Hadoop sirve para más que contar palabras: aquí MapReduce reparte cómputo (CPU-bound) y no datos.
- Lo que aprendimos al correrlo: los directorios no deben existir antes, nDigits debe ser múltiplo de 4 y las cargas salen desiguales.

> **Repositorio**
>
> github.com/Alejandro-Jimenez-Reyes/bda-equipo-5-pi-bbp
> El README trae los pasos para reproducirlo.